

Jupyter Notebook Execution Report

Name: Hany El Atlassi

Date: September 08, 2026

Cell 1: ■ Markdown

TP1 : Text Mining

Hany El Atlassi - CSCC

Cell 2: ■ Markdown

1 Opérations sur des textes bruts

Cell 3: ■ Code

```
import nltk
```

Cell 4: ■ Code

```
from nltk.book import *
```

Output:

```
*** Introductory Examples for the NLTK Book ***  
Loading text1, ..., text9 and sent1, ..., sent9  
Type the name of the text or sentence to view it.  
Type: 'texts()' or 'sents()' to list the materials.  
text1: Moby Dick by Herman Melville 1851  
text2: Sense and Sensibility by Jane Austen 1811  
text3: The Book of Genesis  
text4: Inaugural Address Corpus  
text5: Chat Corpus  
text6: Monty Python and the Holy Grail  
text7: Wall Street Journal
```

text8: Personals Corpus

text9: The Man Who Was Thursday by G . K . Chesterton 1908

Cell 5: ■ Markdown

1.1 Comptages et fréquences

Cell 6: ■ Code

```
f"Number of words in text2: {len(text2)}"
```

Output:

```
'Number of words in text2: 141576'
```

Cell 7: ■ Code

```
f"Taille du vocabulaire: {len(set(text2))}"
```

Output:

```
'Taille du vocabulaire: 6833'
```

Cell 8: ■ Code

```
def lexical_diversity(text):  
    return len(text)/len(set(text))
```

Cell 9: ■ Code

```
print(lexical_diversity(text2),lexical_diversity(text1), lexical_diversity(text6))
```

Output:

```
20.719449729255086 13.502044830977896 7.833333333333333
```

Cell 10: ■ Code

```
text6.count("love")
```

Output:

```
0
```

Cell 11: ■ Markdown

1.2 Comptage de mots dans différents documents

Cell 12: ■ Code

```
from nltk.corpus import brown
news_text = brown.words(categories="news")
fdist = nltk.FreqDist([w.lower() for w in news_text])
modals = ['can', 'could', 'may', 'might', 'must', 'will']
for m in modals:
    print(m + ":", fdist[m])
```

Output:

```
can: 94
could: 87
may: 93
might: 38
must: 53
will: 389
```

Cell 13: ■ Code

```
from nltk.corpus import brown
news_text = brown.words(categories="science_fiction")
fdist = nltk.FreqDist([w.lower() for w in news_text])
modals = ['who', 'what', 'where', 'why', 'when', 'whom', 'whose', 'which']
for m in modals:
    print(m + ":", fdist[m])
```

Output:

```
who: 13
what: 41
where: 15
why: 8
when: 28
```

```
whom: 2
whose: 3
which: 33
```

Cell 14: ■ Code

```
cf = nltk.ConditionalFreqDist((genre, word) for genre in brown.categories() for
word in brown.words(categories=genre))

genres = ['news', 'religion', 'hobbies', 'science_fiction', 'romance', 'humor']
cf.tabulate(conditions = genres, samples = modals)
```

Output:

	who	what	where	why	when	whom	whose	which
news	268	76	58	9	128	8	22	244
religion	100	64	20	14	53	11	9	202
hobbies	103	78	72	10	119	2	14	252
science_fiction	13	27	10	4	21	2	3	32
romance	89	121	54	34	126	5	9	104
humor	48	36	15	9	52	4	8	62

Cell 15: ■ Markdown

1.3 Les stopwords

Cell 16: ■ Code

```
def unusual_words(text):
    text_vocab = set(w.lower() for w in text if w.isalpha())
    english_vocab = set(w.lower() for w in nltk.corpus.words.words())
    unusual = text_vocab.difference(english_vocab)
    return sorted(unusual)
```

Cell 17: ■ Code

```
unusual_words(nltk.corpus.gutenberg.words('austen-sense.txt'))
```

Output:

```
['abbeyland', 'abhorred', 'abilities', 'abounded', 'abridgement', 'abused', 'abuses', 'accents',
```

Cell 18: ■ Code

```
unusual_words(nltk.corpus.nps_chat.words())
```

Output:

```
['aaaaaaaaaaaaaaaa', 'aaahhhh', 'abortions', 'abou', 'abouted', 'abs', 'ack', 'acros', 'actually']
```

Cell 19: ■ Code

```
from nltk.corpus import stopwords
stopwords.words('english')
```

Output:

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are']
```

Cell 20: ■ Code

```
def content_fraction(text):
    stopwords = nltk.corpus.stopwords.words('english')
    content = [w for w in text if w.lower() not in stopwords]
    return len(content)/len(text)
```

Cell 21: ■ Code

```
content_fraction(nltk.corpus.reuters.words())
```

Output:

```
0.735240435097661
```

Cell 22: ■ Markdown

2 Sélection de descripteurs pertinents

Cell 23: ■ Markdown

2.1 Utilisation de WordNet

Cell 24: ■ Code

```
from nltk.corpus import wordnet as wn  
wn.synsets('motorcar')
```

Output:

```
[Synset('car.n.01')]
```

Cell 25: ■ Code

```
wn.synset('car.n.01').lemma_names()
```

Output:

```
['car', 'auto', 'automobile', 'machine', 'motorcar']
```

Cell 26: ■ Code

```
wn.synset('car.n.01').definition()
```

Output:

```
'a motor vehicle with four wheels; usually propelled by an internal combustion engine'
```

Cell 27: ■ Code

```
wn.synset('car.n.01').examples()
```

Output:

```
['he needs a car to get to work']
```

Cell 28: ■ Code

```
motorcar = wn.synset('car.n.01')  
types_of_motorcar = motorcar.hyponyms()  
types_of_motorcar[26]
```

Output:

```
Synset('touring_car.n.01')
```

Cell 29: ■ Code

```
dish = wn.synset('dish.n.01')
```

```
types_of_dishes = dish.hyponyms()
```

Cell 30: ■ Code

```
sorted([lemma.name() for synset in types_of_motorcar for lemma in synset.lemmas()])
```

Output:

```
['Model_T', 'S.U.V.', 'SUV', 'Stanley_Steamer', 'ambulance', 'beach_waggon', 'beach_wagon', 'bus']
```

Cell 31: ■ Code

```
sorted([lemma.name() for synset in types_of_dishes for lemma in synset.lemmas()])
```

Output:

```
['Petri_dish', 'boat', 'bowl', 'butter_dish', 'casserole', 'coquille', 'gravy_boat', 'gravy_holder']
```

Cell 32: ■ Code

```
motorcar.hypernyms()
```

Output:

```
[Synset('motor_vehicle.n.01')]
```

Cell 33: ■ Code

```
paths = motorcar.hypernym_paths()
len(paths)
```

Output:

```
2
```

Cell 34: ■ Code

```
[synset.name() for synset in paths[0]]
```

Output:

```
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrument']
```

Cell 35: ■ Code

```
[synset.name() for synset in paths[1]]
```

Output:

```
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrument
```

Cell 36: ■ Code

```
dish_paths = dish.hypernym_paths()  
[synset.name() for synset in dish_paths[1]]
```

Output:

```
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrument
```

Cell 37: ■ Code

```
right = wn.synset('right_whale.n.01')  
orca = wn.synset('orca.n.01')  
minke = wn.synset('minke_whale.n.01')  
tortoise = wn.synset('tortoise.n.01')  
novel = wn.synset('novel.n.01')
```

Cell 38: ■ Code

```
right.lowest_common_hypernyms(minke)
```

Output:

```
[Synset('baleen_whale.n.01')]
```

Cell 39: ■ Code

```
right.lowest_common_hypernyms(orca)
```

Output:

```
[Synset('whale.n.02')]
```

Cell 40: ■ Code

```
right.lowest_common_hypernyms(tortoise)
```

Output:

```
[Synset('vertebrate.n.01')]
```

Cell 41: ■ Code

```
right.lowest_common_hypernyms(novel)
```

Output:

```
[Synset('entity.n.01')]
```

Cell 42: ■ Code

```
wn.synset('baleen_whale.n.01').min_depth()
```

Output:

```
14
```

Cell 43: ■ Code

```
wn.synset('whale.n.01').min_depth()
```

Output:

```
5
```

Cell 44: ■ Code

```
wn.synset('vertebrate.n.01').min_depth()
```

Output:

```
8
```

Cell 45: ■ Code

```
wn.synset('entity.n.01').min_depth()
```

Output:

```
0
```

Cell 46: ■ Code

```
print(right.path_similarity(minke))
```

```
,right.path_similarity(orca)
,right.path_similarity(tortoise)
,right.path_similarity(novel))
```

Output:

```
0.25 0.16666666666666666 0.07692307692307693 0.043478260869565216
```

Cell 47: ■ Code

```
print("psim(orca,minke)", orca.path_similarity(minke))
```

Output:

```
psim(orca,minke)= 0.14285714285714285
```

Cell 48: ■ Markdown

2.2 Stemming et lemmatisation

Cell 49: ■ Code

```
raw = """DENNIS: Listen, strange women lying in ponds distributing swords is no
basis for a system of government. Supreme executive power derives from a mandate
from the masses, not from some farcical aquatic ceremony."""
tokens = nltk.word_tokenize(raw)
```

Cell 50: ■ Code

```
porter = nltk.PorterStemmer()
[porter.stem(t) for t in tokens]
```

Output:

```
['denni', ':', 'listen', ',', 'strang', 'women', 'lie', 'in', 'pond', 'distirbut', 'sword', 'is',
```

Cell 51: ■ Code

```
lancaster = nltk.LancasterStemmer()
[lancaster.stem(t) for t in tokens]
```

Output:

```
['den', ':', 'list', ',', 'strange', 'wom', 'lying', 'in', 'pond', 'distirbut', 'sword', 'is', 'n
```

Cell 52: ■ Code

```
wnl = nltk.WordNetLemmatizer()
[wnl.lemmatize(t) for t in tokens]
```

Output:

```
['DENNIS', ':', 'Listen', ',', 'strange', 'woman', 'lying', 'in', 'pond', 'distirbuting', 'sword'
```

Cell 53: ■ Markdown

2.3 Tagging

Cell 54: ■ Code

```
text = nltk.word_tokenize("And now for something completely different")
nltk.pos_tag(text)
```

Output:

```
[('And', 'CC'), ('now', 'RB'), ('for', 'IN'), ('something', 'NN'), ('completely', 'RB'), ('differ
```

Cell 55: ■ Code

```
text = nltk.word_tokenize("They refuse to permit us to obtain the refuse permit")
```

Cell 56: ■ Markdown

2.4 Sélection de mots pertinents par tf-idf

Cell 57: ■ Code

```
from nltk.book import *
```

Cell 58: ■ Code

```
text1
```

Output:

```
&lt;Text: Moby Dick by Herman Melville 1851&gt;
```

Cell 59: ■ Code

```
mytexts = nltk.TextCollection([text1,text2,text3])
```

Cell 60: ■ Code

```
print(mytexts.tf('is',text1),mytexts.tf('is',text2),mytexts.tf('is',text3))
```

Output:

```
0.00649875967625058 0.005142114482680681 0.0059646144223036365
```

Cell 61: ■ Code

```
mytexts.idf('companionableness')
```

Output:

```
1.0986122886681098
```

Cell 62: ■ Code

```
mytexts.tf_idf('companionableness',text2)
```

Output:

```
7.759876593971505e-06
```

Cell 63: ■ Code